大数据可视化实战手册

从数据到图表

竞赛培训组

2026 年 3 月 31 日

学习指引

前序课程：确保你已经完成了数据分析（见《大数据分析实战手册》）。
后续课程：恭喜你完成了全部学习！现在可以参加模拟竞赛了。
配套练习：模拟竞赛练习见仓库根目录的《仿真实战练习.md》中的练习 5。

目录

1 前言：什么是数据可视化？ **2**
1.1 环境准备 2

2 第一章：Matplotlib 基础 **3**
2.1 第一个图表：柱状图 3
2.2 折线图 4
2.3 饼图 4
2.4 散点图 5
2.5 多图组合 6

3 第二章：Seaborn 美化 **7**
3.1 安装与基础用法 7
3.2 箱线图 7
3.3 热力图 8
3.4 分布图 8

4 第三章：结合大数据平台的数据可视化 **9**
4.1 步骤 1：从大数据平台导出数据 9
4.2 步骤 2：用 Python 读取并画图 10

5 第四章：端到端实战——从仿真平台到报告 **12**
5.1 步骤 1：从仿真平台获取并分析数据 12
5.2 步骤 2：Python 画图 13
5.3 步骤 3：撰写分析报告 14

6 第五章：Pyecharts 交互式图表 **15**
6.1 安装 15
6.2 柱状图 15
6.3 折线图 16
6.4 饼图 16
6.5 组合图表（一个页面多张图） 17

7 第五章：竞赛报告中的图表规范 **17**

8 附录：Matplotlib 常用参数速查 **18**

1 前言：什么是数据可视化？

用天气预报理解数据可视化

天气预报告诉你”明天温度 25 度，降水概率 80%”。你可能觉得抽象。
但如果展示一张温度曲线图 + 降水概率柱状图，你一眼就能看出”明天下午可能下雨，温度适中”。
数据可视化就是：把枯燥的数字变成直观的图表，让人一眼看出数据背后的信息。

竞赛中的可视化考什么？

根据竞赛评分标准，可视化属于”综合报告”部分（占 30 分）。要求你：

- 用图表展示分析结果（不能只有数字）
- 图表清晰、美观、有标题和标签
- 图表能支持你的分析结论

常用工具：

- **Matplotlib**：Python 最基础的绘图库（必学）
- **Seaborn**：基于 Matplotlib 的美化版（更美观）
- **Pyecharts**：基于 ECharts 的 Python 库（交互式图表，适合报告）

1.1 环境准备

```
# 安装可视化库（在 Python 环境中）
pip install matplotlib seaborn pyecharts pandas numpy

# 验证安装
python3 -c "import matplotlib; print(matplotlib.__version__)"
```

2 第一章：Matplotlib 基础

Matplotlib 是什么？

Matplotlib 是 Python 最基础的绘图库。虽然默认样式不够美观，但它功能全面、控制力强，是所有 Python 可视化的基础。

类比：Matplotlib 就像一支铅笔——最基础，但你能画出任何图形。

2.1 第一个图表：柱状图

```python
import matplotlib.pyplot as plt

# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']  # 或 'Microsoft YaHei'
plt.rcParams['axes.unicode_minus'] = False      # 解决负号显示问题

# 数据
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
sales = [1520, 980, 760, 1340, 890]

# 画柱状图
plt.figure(figsize=(10, 6))          # 设置画布大小（宽10，高6）
plt.bar(categories, sales, color='steelblue')  # 画柱状图
plt.title('各品类销售量')              # 标题
plt.xlabel('品类')                   # X轴标签
plt.ylabel('销售量')                 # Y轴标签
plt.xticks(rotation=45)              # X轴标签旋转45度
plt.tight_layout()                   # 自动调整布局
plt.savefig('bar_chart.png', dpi=150)  # 保存图片
plt.show()                           # 显示图片
```

2.2 折线图

折线图适合什么场景？

折线图适合展示”趋势”——数据随时间的变化。比如：每天的销售趋势、每小时的访问量变化。

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 数据：7天的销售趋势
days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]

plt.figure(figsize=(10, 6))
plt.plot(days, sales, marker='o', color='steelblue', linewidth
    =2)
plt.title('一周销售趋势')
plt.xlabel('日期')
plt.ylabel('销售量')
plt.grid(True, linestyle='--', alpha=0.5)  # 添加网格线
plt.savefig('line_chart.png', dpi=150)
plt.show()
```

2.3 饼图

饼图适合什么场景？

饼图适合展示”占比”——各部分在整体中占多少比例。比如：各品类销售占比、设备类型分布。

```
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 数据
```

```python
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
shares = [30.5, 19.7, 15.2, 26.9, 7.7]

plt.figure(figsize=(8, 8))
plt.pie(shares, labels=categories, autopct='%.1f%%', startangle=90)
plt.title('各品类销售占比')
plt.savefig('pie_chart.png', dpi=150)
plt.show()
```

2.4 散点图

散点图适合什么场景？

散点图适合展示”两个变量之间的关系”。比如：浏览次数和购买次数的关系、价格和销量的关系。

```python
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 模拟数据：浏览次数 vs 购买次数
np.random.seed(42)
views = np.random.randint(100, 1000, 50)
buys = views * 0.15 + np.random.randint(-20, 20, 50)  # 正相关

plt.figure(figsize=(10, 6))
plt.scatter(views, buys, alpha=0.6, color='steelblue')
plt.title('浏览次数 vs 购买次数')
plt.xlabel('浏览次数')
plt.ylabel('购买次数')
plt.grid(True, linestyle='--', alpha=0.3)
plt.savefig('scatter_plot.png', dpi=150)
plt.show()
```

2.5 多图组合

```python
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 创建 2x2 的子图布局
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 图1：柱状图
categories = ['手机', '电脑', '家电', '服饰', '美妆']
values = [1520, 980, 760, 1340, 890]
axes[0, 0].bar(categories, values, color='steelblue')
axes[0, 0].set_title('各品类销售量')

# 图2：折线图
days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]
axes[0, 1].plot(days, sales, marker='o', color='green')
axes[0, 1].set_title('一周销售趋势')

# 图3：饼图
axes[1, 0].pie(values, labels=categories, autopct='%.1f%%')
axes[1, 0].set_title('品类占比')

# 图4：散点图
import numpy as np
x = np.random.randn(50)
y = x * 0.8 + np.random.randn(50) * 0.3
axes[1, 1].scatter(x, y, alpha=0.6, color='red')
axes[1, 1].set_title('散点图')

plt.tight_layout()
plt.savefig('combined_charts.png', dpi=150)
plt.show()
```

3 第二章：Seaborn 美化

Seaborn 是什么？

Seaborn 是基于 Matplotlib 的”美化版”。它默认样式更好看，代码更简洁，特别适合统计数据的可视化。

类比：如果 Matplotlib 是铅笔，Seaborn 就是彩色铅笔——同样的基础，但画出来更好看。

3.1 安装与基础用法

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 设置 Seaborn 主题
sns.set_theme(style='whitegrid')  # 白色背景+网格
```

3.2 箱线图

箱线图适合什么场景？

箱线图适合展示”数据的分布情况”——最大值、最小值、中位数、四分位数。特别适合比较不同组的数据分布。

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# 创建模拟数据
data = pd.DataFrame({
    '品类': ['手机'] * 100 + ['电脑'] * 100 + ['家电'] * 100,
    '价格': list(np.random.normal(3000, 800, 100)) +
            list(np.random.normal(5000, 1200, 100)) +
            list(np.random.normal(2000, 600, 100))
})
```

```python

plt.figure(figsize=(10, 6))
sns.boxplot(x='品类', y='价格', data=data)
plt.title('各品类价格分布（箱线图）')
plt.savefig('boxplot.png', dpi=150)
plt.show()
```

3.3 热力图

热力图适合什么场景？

热力图适合展示”二维矩阵数据”。比如：不同城市不同品类的销售量，用颜色深浅表示数值大小。

```python
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 创建模拟数据：城市 x 品类 的销售矩阵
cities = ['北京', '上海', '广州', '深圳', '杭州']
categories = ['手机', '电脑', '家电', '服饰', '美妆']

data = pd.DataFrame(
    np.random.randint(100, 1000, (5, 5)),
    index=cities, columns=categories
)

plt.figure(figsize=(10, 8))
sns.heatmap(data, annot=True, fmt='d', cmap='YlOrRd')
# annot=True 显示数字，fmt='d' 整数格式，cmap 颜色方案
plt.title('城市 x 品类 销售热力图')
plt.savefig('heatmap.png', dpi=150)
plt.show()
```

3.4 分布图

```python
import seaborn as sns
```

```python
import matplotlib.pyplot as plt
import numpy as np

# 直方图 + 密度曲线
data = np.random.normal(100, 20, 1000)

plt.figure(figsize=(10, 6))
sns.histplot(data, kde=True, bins=30, color='steelblue')
# kde=True 显示密度曲线
plt.title('成绩分布直方图')
plt.xlabel('成绩')
plt.ylabel('人数')
plt.savefig('distribution.png', dpi=150)
plt.show()
```

4 第三章：结合大数据平台的数据可视化

这一章要做什么？

前面学了怎么画图，这一章教你把大数据平台上的数据读出来，然后画图。
流程：大数据平台 SQL 查询 → 导出 CSV → Python 读取 → 画图。

4.1 步骤 1：从大数据平台导出数据

```bash
# 【在 node1 上执行】从 Hive 导出数据为 CSV

# 导出事件类型统计
hive -e "
SELECT event_type, COUNT(*) as cnt
FROM ecommerce.user_behavior
GROUP BY event_type
ORDER BY cnt DESC
" | sed 's/\t/,/g' > /tmp/event_stats.csv

# 导出品类统计
hive -e "
SELECT category,
       SUM(CASE WHEN event_type='view' THEN 1 ELSE 0 END) as
```

```
          views,
       SUM(CASE WHEN event_type='buy' THEN 1 ELSE 0 END) as buys
FROM ecommerce.user_behavior
GROUP BY category
" | sed 's/\t/,/g' > /tmp/category_stats.csv

# 导出时间趋势
hive -e "
SELECT SUBSTR(timestamp,12,2) as hour, COUNT(*) as cnt
FROM ecommerce.user_behavior
GROUP BY SUBSTR(timestamp,12,2)
ORDER BY hour
" | sed 's/\t/,/g' > /tmp/hourly_stats.csv

# 下载到本地（用于画图）
# scp root@node1:/tmp/event_stats.csv ./
```

4.2 步骤 2：用 Python 读取并画图

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
sns.set_theme(style='whitegrid')

# 读取数据
event_df = pd.read_csv('/tmp/event_stats.csv', names=['event_type', 'cnt'])
category_df = pd.read_csv('/tmp/category_stats.csv', names=['category', 'views', 'buys'])
hourly_df = pd.read_csv('/tmp/hourly_stats.csv', names=['hour', 'cnt'])

# 图1：事件类型分布柱状图
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

```

```python
axes[0, 0].barh(event_df['event_type'], event_df['cnt'], color='steelblue')
axes[0, 0].set_title('用户行为类型分布')
axes[0, 0].set_xlabel('次数')

# 图2: 品类浏览量 vs 购买量
x = range(len(category_df))
axes[0, 1].bar(x, category_df['views'], width=0.4, label='浏览', align='center')
axes[0, 1].bar([i+0.4 for i in x], category_df['buys'], width=0.4, label='购买', align='center')
axes[0, 1].set_xticks([i+0.2 for i in x])
axes[0, 1].set_xticklabels(category_df['category'], rotation=45, ha='right')
axes[0, 1].set_title('各品类浏览量 vs 购买量')
axes[0, 1].legend()

# 图3: 每小时活跃趋势
axes[1, 0].plot(hourly_df['hour'], hourly_df['cnt'], marker='o', color='green')
axes[1, 0].set_title('每小时活跃趋势')
axes[1, 0].set_xlabel('小时')
axes[1, 0].set_ylabel('行为次数')

# 图4: 品类饼图
axes[1, 1].pie(category_df['views'], labels=category_df['category'], autopct='%.1f%%')
axes[1, 1].set_title('各品类浏览占比')

plt.tight_layout()
plt.savefig('bigdata_dashboard.png', dpi=150)
plt.show()
```

5 第四章：端到端实战——从仿真平台到报告

这一章要做什么？

做一个完整的端到端案例：从仿真平台获取数据 → 分析 → 画图 → 写报告。这是竞赛中最可能考到的完整流程。

5.1 步骤 1：从仿真平台获取并分析数据

```
# 【在 node1 上执行】从仿真平台获取数据
curl -s "http://163.7.1.176:8889/api/data?n=5000" > /tmp/
    behavior.json
hdfs dfs -put /tmp/behavior.json /user/data/ecommerce/

# 用 Hive 做分析并导出
hive -e "
USE ecommerce;

-- 导出1: 事件类型统计
SELECT event_type, COUNT(*) as cnt
FROM user_behavior GROUP BY event_type
ORDER BY cnt DESC
" > /tmp/event_stats.tsv

hive -e "
USE ecommerce;

-- 导出2: 品类浏览量vs购买量
SELECT category,
    SUM(CASE WHEN event_type='view' THEN 1 ELSE 0 END) as views,
    SUM(CASE WHEN event_type='buy' THEN 1 ELSE 0 END) as buys
FROM user_behavior GROUP BY category
ORDER BY views DESC
" > /tmp/category_stats.tsv

hive -e "
USE ecommerce;

-- 导出3: 每小时活跃趋势
SELECT SUBSTR(timestamp,12,2) as hour, COUNT(*) as cnt
```

```
FROM user_behavior
GROUP BY SUBSTR(timestamp,12,2) ORDER BY hour
" > /tmp/hourly_stats.tsv

# 下载到本地画图
# scp root@node1:/tmp/*.tsv ./
```

5.2 步骤 2：Python 画图

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
sns.set_theme(style='whitegrid')

# 读取数据
event = pd.read_csv('/tmp/event_stats.tsv', sep='\\t', names=['type', 'cnt'])
category = pd.read_csv('/tmp/category_stats.tsv', sep='\\t', names=['cat', 'views', 'buys'])
hourly = pd.read_csv('/tmp/hourly_stats.tsv', sep='\\t', names=['hour', 'cnt'])

# 创建 2x2 仪表盘
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 图1：行为类型柱状图
axes[0, 0].barh(event['type'], event['cnt'], color='steelblue')
axes[0, 0].set_title('用户行为类型分布')

# 图2：品类浏览量 vs 购买量
x = range(len(category))
axes[0, 1].bar(x, category['views'], width=0.4, label='浏览')
axes[0, 1].bar([i+0.4 for i in x], category['buys'], width=0.4, label='购买')
axes[0, 1].set_xticks([i+0.2 for i in x])
```

```python
axes[0, 1].set_xticklabels(category['cat'], rotation=45, ha='right')
axes[0, 1].set_title('各品类浏览量 vs 购买量')
axes[0, 1].legend()

# 图3：每小时活跃趋势
axes[1, 0].plot(hourly['hour'], hourly['cnt'], marker='o', color='green')
axes[1, 0].set_title('每小时活跃趋势')
axes[1, 0].set_xlabel('小时')

# 图4：品类饼图
axes[1, 1].pie(category['views'], labels=category['cat'], autopct='%.1f%%')
axes[1, 1].set_title('品类浏览占比')

plt.suptitle('电商平台用户行为分析报告', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('analysis_report.png', dpi=150, bbox_inches='tight')
plt.show()
```

5.3 步骤 3：撰写分析报告

竞赛报告结构模板

1. **问题描述**（1 段）：我们分析了什么问题，为什么重要
2. **数据说明**（1 段）：数据来源、数据量、字段含义
3. **分析方法**（1 段）：用了什么工具，做了哪些分析
4. **分析结果**（核心，配图表）：每张图配 2-3 句解读
5. **结论建议**（1 段）：根据分析结果给出建议

常见扣分点

- 只有数字没有图表（图表是必考项）
- 图表没有标题和标签（不专业的表现）

- 只展示数据不做解读（分析不是数据罗列）
- 没有结论建议（分析的目的是给建议）
- 报告排版混乱（用 Word 排版，字号统一）

6 第五章：Pyecharts 交互式图表

Pyecharts 是什么？

Pyecharts 是基于百度 ECharts 的 Python 库，能生成交互式 HTML 图表（鼠标悬停显示数据、可缩放）。
适合场景：竞赛报告需要展示效果好、有交互的图表。

6.1 安装

```
pip install pyecharts
```

6.2 柱状图

```
from pyecharts.charts import Bar
from pyecharts import options as opts

# 数据
categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
sales = [1520, 980, 760, 1340, 890]

# 创建柱状图
bar = Bar()
bar.add_xaxis(categories)
bar.add_yaxis('销售量', sales)
bar.set_global_opts(
    title_opts=opts.TitleOpts(title='各品类销售量'),
    xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=45))
)

```

```python
# 生成 HTML 文件
bar.render('bar_chart.html')
print('图表已生成: bar_chart.html (用浏览器打开)')
```

6.3 折线图

```python
from pyecharts.charts import Line
from pyecharts import options as opts

days = ['周一', '周二', '周三', '周四', '周五', '周六', '周日']
sales = [120, 135, 128, 142, 158, 210, 195]

line = Line()
line.add_xaxis(days)
line.add_yaxis('销售量', sales, is_smooth=True)
line.set_global_opts(title_opts=opts.TitleOpts(title='一周销售趋势'))

line.render('line_chart.html')
```

6.4 饼图

```python
from pyecharts.charts import Pie
from pyecharts import options as opts

categories = ['手机数码', '电脑办公', '家用电器', '服饰鞋包', '美妆护肤']
shares = [30.5, 19.7, 15.2, 26.9, 7.7]

pie = Pie()
pie.add('', [list(z) for z in zip(categories, shares)])
pie.set_global_opts(title_opts=opts.TitleOpts(title='品类占比'))
pie.set_series_opts(label_opts=opts.LabelOpts(formatter='{b}: {c}%'))

pie.render('pie_chart.html')
```

6.5 组合图表（一个页面多张图）

```python
from pyecharts.charts import Bar, Line, Pie, Grid
from pyecharts import options as opts
from pyecharts.charts import Page

# 创建多个图表
bar = Bar()
bar.add_xaxis(['手机', '电脑', '家电', '服饰', '美妆'])
bar.add_yaxis('销量', [1520, 980, 760, 1340, 890])
bar.set_global_opts(title_opts=opts.TitleOpts(title='品类销量'))

line = Line()
line.add_xaxis(['周一', '周二', '周三', '周四', '周五'])
line.add_yaxis('趋势', [120, 135, 128, 142, 158])
line.set_global_opts(title_opts=opts.TitleOpts(title='销售趋势')
    )

pie = Pie()
pie.add('', [('手机', 30), ('电脑', 20), ('家电', 15), ('服饰',
    27), ('美妆', 8)])
pie.set_global_opts(title_opts=opts.TitleOpts(title='品类占比'))

# 组合到一个页面
page = Page()
page.add(bar, line, pie)
page.render('dashboard.html')
print('仪表盘已生成: dashboard.html')
```

7 第五章：竞赛报告中的图表规范

报告图表的黄金法则

1. **每个图表必须有标题**：标题要说明”这张图在说什么”
2. **X 轴和 Y 轴必须有标签**：标明”这是什么维度”和”单位是什么”
3. **图例必须有**：如果有多个数据系列，必须有图例说明
4. **不要过度装饰**：颜色不超过 5 种，不需要 3D 效果

5. **图表要有结论**：在图表下方写 1-2 句话说明你从图中看到了什么

常见图表选择指南

- **比较**不同类别 → 柱状图
- **趋势**随时间变化 → 折线图
- **占比**各部分比例 → 饼图
- **分布**数据集中在哪里 → 直方图、箱线图
- **关系**两个变量关联 → 散点图
- **矩阵**两个维度的值 → 热力图

配套练习

配套练习见 `仿真实战练习.md`（练习 4、练习 5），竞赛模拟见练习 5。

8 附录：Matplotlib 常用参数速查

参数	说明
figsize=(w, h)	画布大小
title=' 标题'	图表标题
xlabel/ylabel	坐标轴标签
color='blue'	颜色
marker='o'	数据点标记
linewidth=2	线宽
alpha=0.6	透明度
grid(True)	显示网格
legend()	显示图例
tight_layout()	自动调整间距
savefig('f.png', dpi=150)	保存图片